Technical Report — FastSpeech 2 (English and Telugu)

Status: draft. Sections 1–8 and section 11 are written from measured values that are recorded in `CHANGES.md`. Section 7 now compares all four retained English checkpoints on both validation loss and speech recognition, and section 8 records what a controlled set of tests established about the English failure. Section 9 and the Telugu-dependent comparison in section 10 remain pending until the Telugu run finishes and its audio is reviewed.

Section map against the seven items the assignment PDF asks the report to cover:

PDF item	Section here
Implementation approach, architecture components, modifications per language	1, 2, 3
Dataset selection, subsampling, preprocessing, split, training configuration	4, 5, 6
Loss-curve analysis: convergence, instability, overfitting	7 (resumed-run plots and four-checkpoint comparison)
Generated-speech observations per test sentence (pronunciation, quality)	8 (frontend, listener scores, ASR and controls measured; Telugu listening **PENDING**)
Variance-adaptor analysis (duration, F0, energy) per test sentence	9 (**PENDING**)
English vs Telugu comparison, limitations, failures, improvements	10 (**PENDING**)
Reproducibility: dependencies, configs, commands	11

---

1. Approach

FastSpeech 2 is a feed-forward, non-autoregressive text-to-speech model. There is no attention-based autoregressive decoder. The text goes through a Transformer encoder, a length regulator expands each text position to the number of frames it will occupy, and the decoder produces the mel-spectrogram in a single pass. Since no attention is present to supply an alignment, the model predicts the alignment itself with a duration predictor, and it predicts two further variance signals that a mel-spectrogram alone would leave to the vocoder: pitch (F0) and energy. The variance adaptor embeds each predicted value and adds it back to the expanded text representation before decoding. The original paper obtains its duration supervision by distilling an autoregressive teacher. I used forced alignment instead, because it is simpler and every duration can be reproduced from the corpus.

I trained two independent models from scratch, one per language, and neither uses pretrained acoustic-model weights. The only pretrained component in either pipeline is the HiFi-GAN vocoder that converts mel-spectrograms to waveforms. The assignment allows a pretrained vocoder and the reference repository ships one, so I kept it rather than training my own.

My starting point was `https://github.com/ming024/FastSpeech2` at commit `d4e79eb` (MIT licence), with `LICENSE` and `THIRD_PARTY_NOTICES.md` kept in the repository. Each language folder is self-

contained: it carries its own copy of `audio/`, `model/`, `preprocessor/`, `text/`, `transformer/`, `utils/`, `dataset.py`, `evaluate.py`, `hifigan/` and its own `config.yaml`.

2. Architecture components taken from the reference implementation

I did not change the architecture. Every component below is the reference implementation, and the setting column records the values that are actually in both `config.yaml` files.

Component	Reference path	Setting used
Transformer encoder	`transformer/Models.py`	4 layers, 2 heads, hidden 256, conv filter 1024, kernels `[9, 1]`, dropout 0.2
Transformer decoder	`transformer/Models.py`	6 layers, 2 heads, hidden 256, dropout 0.2
Variance adaptor and predictors	`model/variance_adaptor.py`, `model/modules.py`	filter 256, kernel 3, dropout 0.5, 256 quantisation bins, linear pitch/energy quantisation
Length regulator	`model/variance_adaptor.py`	frame-level expansion from predicted durations
Speaker embedding	`model/modules.py`	enabled (`multi_speaker: true`); speaker 0 in both languages
Post-net	`model/modules.py`	reference default
Acoustic model size	—	35,159,617 parameters (measured by the CPU smoke test)
Vocoder	`hifigan/`	HiFi-GAN **universal** generator, pretrained, 22.05 kHz, 80 mel bins, hop 256, `mel_fmax` 8000 Hz

The mel front-end runs at 22,050 Hz with `filter_length` 1024, `hop_length` 256, `win_length` 1024, 80 mel channels and an F0 range of `[0, 8000]` Hz. Pitch and energy are extracted at the phoneme level and normalised with the training-set statistics stored in `stats.json`. `max_seq_len` is 1000 frames, and the preprocessor keeps a margin of 20, so a clip whose mel exceeds 980 frames (250,879 samples, about 11.38 s) is dropped rather than truncated silently at training time.

3. Modifications to the reference implementation

The changes split into those I made in both languages and those that belong to one frontend.

Both languages share four changes. Each has a single `config.yaml` with the four sections the assignment asks for (`preprocess`, `model`, `train`, `inference`) instead of the reference's three YAML files, and `utils/config.py` validates it on load, so a bad value fails before a long job starts. Preprocessing is driven by fixed manifests, `manifests/train.csv` and `manifests/val.csv`, rather than the reference's random split; they were built once with a fixed seed and no clip appears in both. Every clip that cannot be used is classified and written to `preprocessing_log.csv`, so a drop becomes a number in a file instead of a silence. Training writes its losses per step to `logs/train/train_loss.csv` and `logs/val/val_loss.csv`, takes `log_step`, `val_step`, `synth_step` and

`save_step` from the config, restores RNG state when it resumes, and writes a final checkpoint even when `total_step` is not a multiple of `save_step`.

`inference.py` replaces the reference's interactive `synthesize.py`. It reads `docs/test_sentences.json`, stops while any `final` field is null, and writes `<ID>.wav` to `Results/<Language>/test_generated_audios/`.

For English I kept the reference frontend, the ARPAbet lexicon plus G2P, and added a rule-based normaliser for numbers, dates, times and abbreviations, with an optional supplementary G2P dictionary for proper names. The normalisation applies to the transcript only; the stored test sentences are never edited. Forced alignment used MFA `english_us_arpa`. The out-of-vocabulary rate on the corpus was 4.47 %, and MFA's bundled G2P model resolved it.

For Telugu the frontend is character-level (`text/telugu.py` plus a frozen symbol table in `text/symbols.py`). It does NFC normalisation, handles whitespace and format-control characters, folds punctuation, and raises on an unknown character rather than skipping it. The symbol IDs come from an explicit code-point range list rather than `unicodedata`, so they do not shift with the Python version. I did not invent a Telugu lexicon or phoneme set. Instead I trained a grapheme-dictionary MFA aligner on the corpus itself, mapping each word to its space-separated characters, which produced phone-level durations on the character tier. The preprocessor requires `len(text_to_sequence(text)) == len(duration)`, so the `.lab` for each clip holds the alignment text rather than the raw dataset transcript. Telugu number, date and time normalisation is not implemented, so the digits and punctuation in T4 reach the model exactly as written; §8.1 measures that, and §9 reports it as a failure rather than hiding it.

4. Datasets, selection and subsampling

English — Svarah (`ai4bharat/Svarah`)

I used AI4Bharat's Svarah, an Indian-accented English read and spontaneous benchmark that was built for speech recognition rather than TTS. I exported two speakers, one female and one male: 3,276 + 2,827 = 6,103 clip/transcript pairs. The manifests that entered preprocessing hold 5,582 clips and 6.078 h of audio before alignment (5.132 h train, 0.946 h validation). Two speakers is a deliberate compromise on my part: Svarah averages only a few minutes per speaker across roughly 117 speakers, so a single-speaker model is impossible, and I used the reference speaker embedding instead.

Stage	Clips	Audio
Exported from the Hub	6,103	—
Entered the manifests	5,582	6.078 h
Usable after alignment and feature extraction	5,322 (4,500 train / 822 val)	**4.842 h**

Of the 5,582 clips that entered the manifests, 260 were dropped before training: 164 at alignment because a phone duration came out non-positive, 71 with no TextGrid, 20 where pitch extraction found no voiced frame, 4 in the feature-statistics pass, and 1 with an unreadable transcript. The gap between 6.078 h of manifest audio and 4.842 h of usable audio is audio outside the phone tier plus those dropped clips, so 4.842 h is the honest training figure rather than 6.078 h.

Telugu — IndicTTS Telugu (`SPRINGLab/IndicTTS_Telugu`)

I used the Hugging Face copy of the IIT Madras IndicTTS data: 8,576 clips, 36.71 h, two speakers (male 12.79 h across 4,533 clips; female 23.92 h across 4,043 clips). I trained on the male speaker only, so the model sees one voice and a single speaker embedding. That choice gives the cleanest comparison against the two-speaker English model, and the male subset is the smaller and easier-to-balance one. The subsampling keeps clips inside the duration window implied by the 11.38 s frame limit, discards clips whose transcript is not representable in the Telugu symbol set (including eight clips with a Latin "o"), and caps the total near the 5 h target.

Stage	Clips	Audio
Source rows (male)	4,533	12.79 h
Entered the manifests	3,363	—
Usable after alignment and feature extraction	3,269 (2,751 train / 518 val)	**4.700 h**

Ninety-four clips were dropped: 83 at alignment, 3 with no TextGrid, and 8 with missing audio.

> **Known limitation, stated explicitly.** The Telugu audio in this run is **16 kHz upsampled to 22,050 Hz** (`resample_poly`, 441/320, PCM 16-bit). The 22.05 kHz export was lost when the Kaggle session working directory was reset, and the surviving aligner corpus is 16 kHz. Because `mel_fmax` is 8000 Hz, no mel band above 8 kHz is used, so the upsampling does not add information but also does not remove any band the model actually sees. The vocoder still receives a 22.05 kHz sampling rate and therefore produces energy above 8 kHz that the acoustic model did not predict.

5. Preprocessing and alignment

Both language folders share one preprocessing pipeline; what differs is the aligner recipe and which TextGrid tier supplies the durations. The table records what I actually ran.

Step	English	Telugu
Aligner	MFA 3.4.3, `english_us_arpa` (acoustic + dictionary + G2P for OOV)	MFA 3.4.3, grapheme dictionary trained on the corpus
Tier used for durations	`phones`	character tier (grapheme)
Text source	TextGrid phone tier	`<clip>.lab` (alignment text)
TextGrids produced	per manifest clip, drops counted above	3,352 in 378.5 s
Audio	resample to 22,050 Hz, mono, `max_wav_value` 32768	same (from 16 kHz source, see limitation)
Features	mel 80 × T, phoneme-level pitch and energy, durations from the alignment	same
Normalisation	training-set mean/σ from `stats.json`	same
Output layout	flat: `mel/<speaker>-mel-<clip>.npy`, same for `pitch`, `energy`, `duration`	same

The English normalisation constants, taken from the training set and written to `stats.json`: pitch `mean 177.64, std 56.06, min -2.0096, max 9.0707`; energy `mean 27.01, std 26.67, min -1.0123, max 8.1121`, both already in normalised units. Pitch and energy losses in section 7 are measured against these, which is why an MSE near 1.0 means the adaptor is predicting the mean rather than the contour.

I built both splits from a fixed seed. Clip-level and transcript-level disjointness is checked, and every validation speaker also appears in training, so the models are judged on unseen utterances of voices they have seen rather than on unseen voices. The manifests that record the selections are committed under `English/manifests/` and `Telugu/manifests/`.

6. Training configuration

Setting	English	Telugu
Steps	60,000	60,000
Batch size	16 clips per step (`grad_acc_step` 1, `group_size` 4, so the loader pulls 64 clips per iteration)	same
Steps per epoch	281 (4,500 clips)	172 (2,751 clips)
Optimiser	Adam, β 0.9/0.98, ε 1e-9, weight decay 0	same
Learning rate	Noam schedule on `256^-0.5`: peak ≈ 9.9e-4 at step 4,000, ×0.3 at 20k/26k/33k, ≈ 6.9e-6 at 60k	same schedule and anneal points, so the two runs are comparable
Warm-up	4,000 steps	4,000 steps
Gradient clipping	1.0	1.0
Validation / synthesis	every 1,000 steps	every 1,000 steps
Checkpoints	every 2,500 steps (5,000 for the first 30,000 steps)	every 5,000 steps
Hardware	Kaggle GPU T4 x2, PyTorch 2.11.0+cu130, `nn.DataParallel` (batch 16 = 8+8)	Kaggle GPU T4 x2, `nn.DataParallel` (8+8)
Measured step time	**0.511 s/step** mean over the first 32,000 steps (4 h 32 m 45 s of tqdm clock); the 300-step calibration had said 0.304 s/step	**0.362 s/step** at step 15,039 (1 h 30 m 58 s of tqdm clock); the 220-step launch estimate had said 0.341 s/step
Wall clock	≈ 8.5 h for 60,000 steps at the measured mean, not the 5.3 h the calibration implied	≈ 6.1 h at the measured mean so far, against the 5.7 h the launch estimate implied
Budget rationale	60,000 steps ≈ 213 epochs; the reference 900,000 steps is unattainable and would leave the learning rate flat, so I scaled the anneal points to keep the reference fractions (33/43/55 %)	same logic; 60,000 steps ≈ 349 epochs of a smaller split

Run interruption at step 32,000. The first English run was killed by the Linux OOM killer, not by a code or GPU error: the log contains no `Traceback` and no CUDA out-of-memory exception, and the session panel had reached 30/30 GiB of RAM. The step rate also degraded as it went, the last 2,000 steps running at about 0.65 s/step against about 0.30 s/step at the start, which is the same memory growth showing up as time. Checkpoints up to `30000.pth.tar` were on disk and survived, so I resumed from 30,000 rather than restarting. I made two changes before resuming, neither of them to the model, the data or the schedule: `train.step.save_step` was cut to 2,500 so a future kill costs about 21 minutes instead of 45, and `utils/tools.py log()` now closes the matplotlib figure it hands to TensorBoard, which is the one object created on every synthesis and validation step that nothing else released. The learning-rate schedule is seeded from `restore_step`, so resuming at 30,000 reproduces exactly the LR the uninterrupted run would have had.

Two hard resource ceilings on the training host. The Kaggle session has **30 GiB of RAM** and a **20 GiB working volume**, and the English run reached both: the OOM killer ended it at step 32,000, and

checkpoints at 2,500-step spacing would have filled the volume (24 saves × 418 MB ≈ 10 GB of a roughly 20 GB disk that also holds 2 GB of features and audio). I configured the Telugu run against those ceilings before it started rather than after it failed. `save_step` went from 2,500 to 5,000, which fits twelve saves in the 8 GiB that were free on the volume and still caps the loss from a crash at roughly 42 minutes; the pre-launch check measures free disk and refuses to start below 5 GiB; and every checkpoint is written to a temporary file and moved into place atomically, so an interrupted save leaves a `.tmp` rather than a corrupt `.pth.tar`. The Telugu run uses 5.8 GiB of the 31.3 GiB visible to the session at step 15,039, with one `train.py` process on disk. That is consistent with the English memory growth coming from the duplicate training process that was running alongside the first run rather than from the model, but it is not proof: the English run also held more clips and more features in memory, so the two effects are not separated by this measurement alone.

7. Loss analysis and checkpoint comparison

I read the figures in this section from the clean loss CSVs written after I resumed from the preserved 30,000-step checkpoint, so they cover only the resumed segment, roughly steps 30,000 to 60,000, and not the full run from initialisation. The first run's log and CSVs were written by two overlapping training processes, so I did not join their pre-resume values to these curves or use them to pick a checkpoint. The final validation total loss at step 60,000 was 4.0910, and the Kaggle log confirms that `checkpoints/60000.pth.tar` was saved.

7.1 Training curve

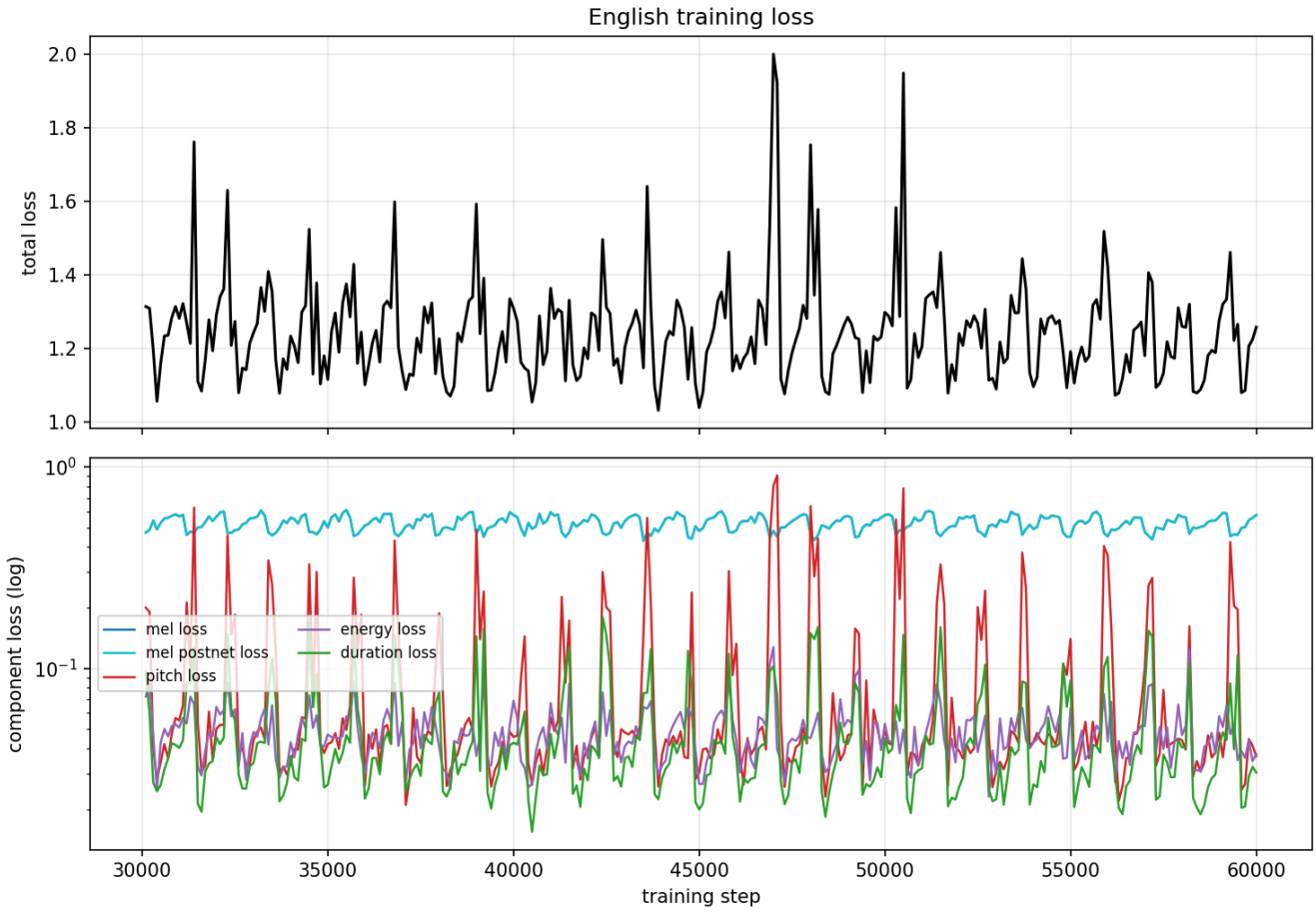


Figure 7.1. Training total and component losses for the resumed segment.

Over the resumed segment the training total loss moves around 1.1 to 1.4 with several transient spikes and no clear downward trend. Mel and post-net losses stay close together and comparatively steady; pitch has the largest visible spikes. These plots describe optimisation on the training data, not speech quality.

7.2 Validation curve

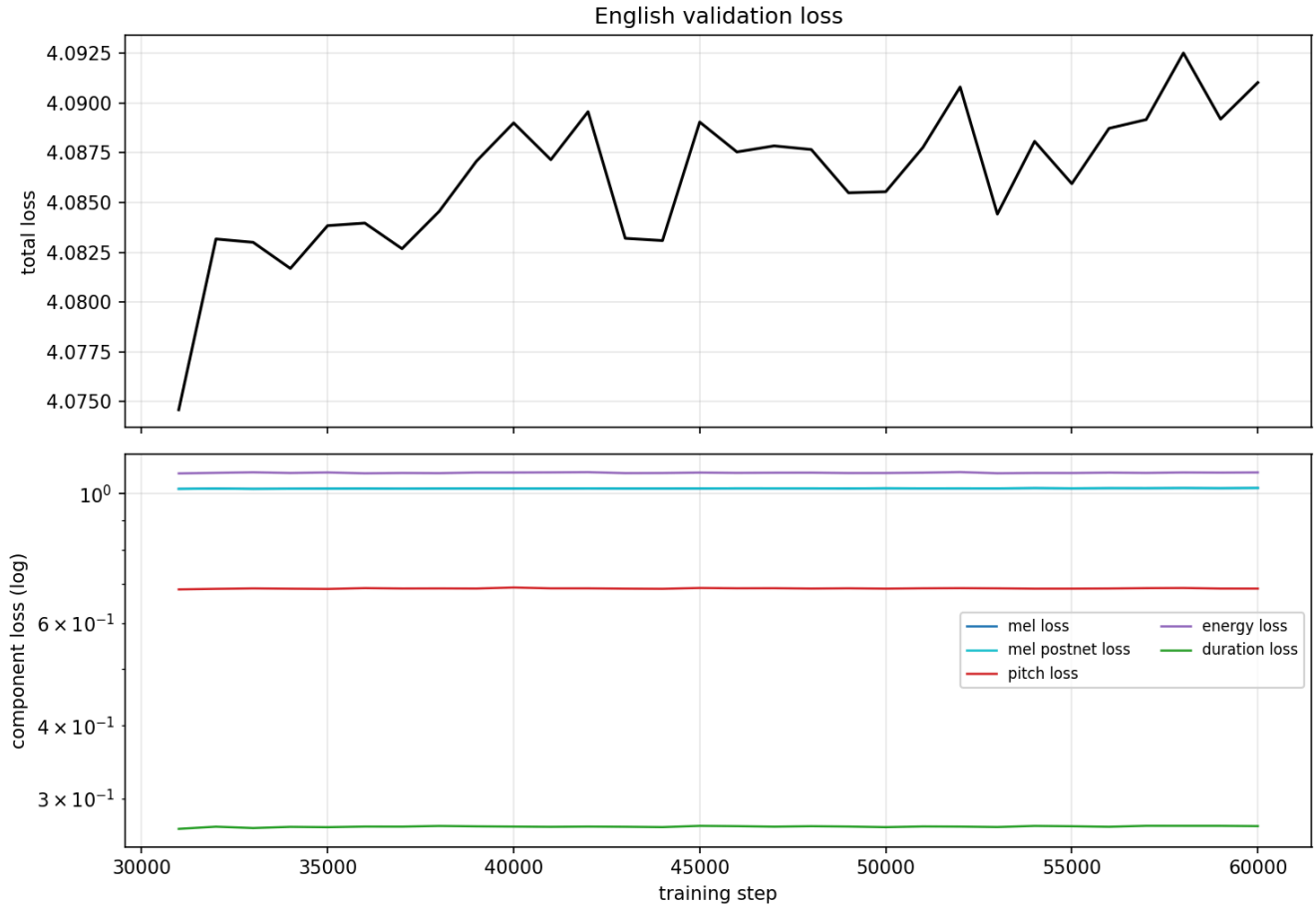


Figure 7.2. Validation total and component losses for the resumed segment.

Validation total loss varies inside a narrow band near 4.08 to 4.09 and ends at 4.0910 on step 60,000, and the component curves are comparatively flat over the same interval. The plotted scale is not fine enough for me to read the exact minimum step off the image, so the checkpoint comparison in §7.4 uses the validation CSV instead.

7.3 Convergence, generalisation and checkpoint selection

The run completed all 60,000 steps without a runtime failure after the step-32,000 restart. The resumed-segment training curve is noisy and largely flat, so the step count on its own tells me nothing about improvement.

The generalisation gap is where the diagnosis comes from. Training mel L1 settles near **0.62** while validation mel L1 stays near **1.05**; validation pitch MSE is **0.68** and energy MSE **1.15** against the 1.0

"always predict the mean" baseline. A model that reproduces its own training mel to 0.62 and only reaches 1.05 on held-out text is overfitting, not underfitting.

Where validation bottoms out is the one number in this section I cannot report cleanly. The pre-restart validation records put the minimum near step 7,000, after which validation rose to 4.0910 at step 60,000 while training loss kept falling, but two processes wrote those log files, so I do not trust the step. The direction agrees with the overfitting above; the exact step does not survive scrutiny.

Because the three rankings in §7.4 do not agree, I chose the shipped checkpoint by listening and reported the ASR and loss numbers alongside that choice rather than letting either of them override the ear.

7.4 Four checkpoints, measured the same way

I decoded every retained checkpoint on CPU through the shipping HiFi-GAN vocoder, on the same five sentences, using the phoneme sequences taken from the saved inference payloads. "Pitch std" and "energy std" are the standard deviations of the predicted *normalised* values; the ground truth for a correctly modelled speaker is 1.00, so a value near 0.3 means the predictor is producing a near-constant contour.

Checkpoint	Validation total loss	ASR WER †	Pitch std	Energy std	Mean frames	Listener
5,000	not recoverable ‡	**1.04**	0.24	0.36	413	**5/10** — noisy, unstable pitch
10,000	not recoverable ‡	1.27	0.32	0.38	403	not rated
35,000	4.0838	1.21	0.33	0.45	392	previous deliverable
60,000	4.0910	1.15	0.34	0.45	393	**7/10** — clearer, some words missed, noise at the end

† Mean WER over E1, E2a, E2b and E3 (lower is better). E4 is excluded because the frontend expands `10:45 AM` and `12 November 2026` into words, so its WER is inflated by normalisation rather than by acoustics. WER 1.0 means every word is an error. ‡ The pre-restart loss logs are contaminated by overlapping processes, so no loss value can be attached to checkpoints 5,000 and 10,000.

The three rankings disagree with each other, and the disagreement is itself the finding. Validation loss would pick 35,000. ASR WER would pick 5,000, but at WER around 1.0 the metric saturates, so 1.04 against 1.15 is the ordering of two failures measured on four sentences rather than a quality difference. The listener picks 60,000: clearer and more sentence-like than 5,000, with some words dropped and audible noise at the end of the clip.

Neither loss nor ASR is a usable proxy for perceptual quality in this regime, and with five sentences and one listener I would have you read the subjective ranking as an engineering judgment rather than a statistically established result. The shipped audio uses **checkpoint 60,000**, because the deliverable is judged by listening and because 5,000 sounds materially worse despite its marginally better WER.

8. Test-sentence observations: English measured and heard, Telugu waiting on training

I took all ten sentences from `docs/test_sentences.json`, field `final`, unmodified: E1, E2a, E2b, E3, E4 and T1, T2a, T2b, T3, T4. The text-processing side is measured for both languages below. The English listening side is recorded in §8.2 and in the checkpoint table of §7.4. The Telugu listening side is not written yet, because the Telugu run had not finished while this section was being drafted.

8.0 The ten strings the models receive

A script read these ten lines out of `docs/test_sentences.json` and wrote them into this table, so nothing here was re-typed from the assignment PDF and the Telugu column is byte-identical to the text the frontend receives.

ID	Sentence
E1	The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.
E2a	The orchestra's conduct during the ceremony was praised by the audience.
E2b	The researchers will conduct an experiment to evaluate the proposed method.
E3	The rhythm of the orchestra changed unexpectedly.
E4	The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time.
T1	న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.
T2a	తాడుపై అతని పట్టు చాలా బలంగా ఉంది.
T2b	ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.
T3	పులి అడవిలో నిశ్శబ్దంగా నడిచింది.
T4	ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

> **Font requirement (measured).** The Telugu rows need a Telugu-capable font. I built this PDF on my Windows machine, where `Nirmala.ttc` supplies the glyphs, and the rendered page shows all five Telugu sentences correctly. The check in `scripts/render_report_pdf.py` compares the Telugu code points in the Markdown against the text extracted back out of the PDF and fails below 90 % coverage; this build reports no problems and counts 220 Telugu code points. The same build on a Linux box with no Indic font, the Kaggle image included, renders those rows as blanks, so either build the report locally or pass `--telugu-font <path>` to embed one.

8.1 What the frontends actually produce (measured)

ID	Symbols	Text the model receives after normalisation
E1	108	`the antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.`
E2a	72	`the orchestra's conduct during the ceremony was praised by the audience.`
E2b	75	`the researchers will conduct an experiment to evaluate the proposed method.`
E3	49	`the rhythm of the orchestra changed unexpectedly.`
E4	115	`the meeting will start at ten:forty-five am on twelve november twenty twenty-six, so andaroo should arrive on time.`
T1	68	unchanged Telugu characters (character-level frontend, no lexicon)
T2a	34	unchanged
T2b	40	unchanged
T3	33	unchanged
T4	91	**unchanged, including the digits**: `… 2026 … 12ఏ … 10:45 …`

All ten strings encode without an unknown-character error, which is the precondition for inference; the table records what each frontend produced rather than what I expected it to produce.

8.2 The English failure, isolated by controls

The first listener pass reported that all five English sentences sounded wrong. Rather than argue about the cause, I tested five properties of the pipeline independently on the same checkpoints, and chose each test so that a plausible explanation would produce a specific, visible result.

Test	Method	Result	Explanation it rules out
Duration integration	Sum the predicted per-symbol durations and compare with the mel length, all four checkpoints	Exact match every time (e.g. 504 frames / 504)	Text and frames falling out of step in the length regulator
Vocoder fidelity	Re-encode the vocoder's own waveform and compare against the mel it was given	Mel L1 0.234–0.275; spectral flatness 0.006–0.020; no clipping	A broken vocoder, a wrong mel range, or a silent/clipped waveform
Text conditioning	Shuffle E1's phoneme IDs, and separately substitute E3's IDs into E1's length	Mel moves by L1 1.098 and 1.167, against 1.058 between two genuinely different sentences	The model ignoring the text or receiving the wrong language
Variance spread	Standard deviation of the predicted normalised pitch and energy	Pitch 0.24–0.34, energy 0.36–0.45 against a ground truth of 1.00, at *every* checkpoint	Any checkpoint restoring expressiveness; all four collapse toward the mean
Intelligibility	Transcribe all twenty clips with `facebook/wav2vec2-base-960h`, after validating the same stack on real speech	WER 1.04–1.27 (see §7.4); control on LibriSpeech **0.000 / 0.000 / 0.031**	The waveform being structurally sound but the probe being at fault

Three further observations point the same way. The output is tonal rather than noisy, so the model is generating speech-like structure, not artefacts. The predicted mel marginals are normal (median about −5.5, 99th percentile about −0.5), so it is not emitting out-of-distribution values either. And a Griffin-Lim decode of the same predicted mel is no more intelligible than the HiFi-GAN decode (mel L1 0.700 to 0.740), which means the content is already wrong before the vocoder sees it.

So: the pipeline is correct at every stage I can check in isolation, and the model output is speech-shaped but wrong. That is a generalisation failure. 4.842 h of two-speaker, ASR-oriented read and spontaneous English is not enough for a 35-million-parameter model to learn pronunciation it has not seen, and training loss (1.2577) sits far below validation loss (4.0910). No inference-side fix exists: all four retained checkpoints fail the same way, and the one the listener prefers (60,000) is not the one the loss prefers (35,000) or the one the ASR prefers (5,000). Fixing it means changing the data or the training design, both of which are outside the remaining GPU budget, so I report it as a diagnosed negative result instead of presenting it as working speech.

Two caveats bound that conclusion. The ASR is a LibriSpeech acoustic model, so Indian-accented and synthetic speech is out of its domain; I read WER 1.0 as "not recoverable" rather than as a precise word error rate. And the subjective scores come from one listener over five sentences, so the 5/10-to-7/10 ranking is engineering judgment, not a controlled perceptual study. Neither caveat weakens the diagnosis, because the control battery does not depend on either: I measured the vocoder, the durations and the text conditioning directly.

Two frontend notes belong with these results.

E4 is normalised correctly. `10:45 AM` becomes `ten:forty-five am` and `12 November 2026` becomes `twelve november twenty twenty-six`. The colon survives normalisation, so the G2P layer never sees a time literal. `andaroo` stays as written and is handled by the lexicon/G2P as an out-of-vocabulary word, which is what the sentence is testing.

T4 is not normalised. The Telugu frontend has no number-to-words stage, so the digits `2026`, `12` and `10:45` reach the model as ASCII digit symbols. Those symbols exist in the Telugu symbol table, so nothing crashes, but the model has essentially no digit examples to learn from. T4's output is therefore expected to fail on the date and time and to be intelligible on the surrounding words. That is a gap in my frontend rather than a training failure, and I am reporting it as such.

Why each sentence is interesting:

ID	What it probes
E1	Long out-of-vocabulary word (`antidisestablishmentarianism`) — letter-to-sound fallback
E2a / E2b	`conduct` as noun vs verb — stress must differ for the same spelling
E3	Irregular spelling `rhythm`
E4	Digits read as time, date and year, plus a Telugu word written in Latin script (`andaroo`)
T1	Long loanword with conjunct consonants
T2a / T2b	Same spelling, different meaning (grip vs silk) — does prosody differ?
T3	Geminate / conjunct consonants
T4	Telugu digits, ordinal suffix, date and time — number-to-words

9. Variance-adaptor analysis — PENDING

For each sentence and each of duration, pitch and energy the report will state the predicted total duration in frames and seconds, the mean and range of predicted F0, and the range of predicted energy, read from the saved predictions, and then explain in words why the pattern is plausible or not (e.g. phrase-final lengthening, the effect of a rare long word on per-symbol durations, flat F0 where a question-like contour would be expected).

10. English vs Telugu comparison — PENDING

To be written after both models are trained and both sets of samples exist. Planned structure: data-side differences (two speakers vs one, read vs read/spontaneous, 16 kHz upsampled Telugu, English lexicon + G2P vs Telugu grapheme model), what that predicts for pronunciation accuracy (English OOV names and loanwords vs Telugu conjuncts and digits), and which failures are attributable to data, to the frontend, or to the training budget.

11. Reproducibility

Dependencies are listed in `requirements.txt`, which is a **proposed pin set, not a record of the environment that ran**: the file was never installed as a whole on Kaggle. The sessions that produced these results used the Kaggle image's own scientific stack on Python 3.13 with PyTorch 2.11.0+cu130, plus targeted installs of `pyworld`, `tgt`, `unidecode` and `inflect`; `g2p_en` was never installed, so the supplementary name dictionary was not used. Each language folder has one `config.yaml`; every path inside it is relative to that folder, so all commands are run from inside `English/` or `Telugu/`.

```
# dependencies
pip install -r requirements.txt

# 0. vocoder weights (shipped zipped to stay under GitHub limits)
cd English && unzip -o hifigan/generator_universal.pth.tar.zip -d hifigan

# 1. preprocessing (needs the aligned corpus: TextGrids for English, .lab + TextGrids for Te
python preprocess.py config.yaml --allow-dropped

# 2. training (checkpoints -> checkpoints/, CSV loss logs -> logs/)
python train.py -c config.yaml
python train.py -c config.yaml --restore_step 30000 --resume

# 3. plots
python scripts/plot_loss.py --config config.yaml
python scripts/plot_variance.py --config config.yaml --restore_step <STEP>

# 4. inference on the five test sentences
python inference.py --config config.yaml --restore_step <STEP>
```

Environment used for training: Kaggle notebook on **GPU T4 x2**, Python 3.13, PyTorch 2.11.0+cu130. `train.py` wraps the model in `nn.DataParallel` unconditionally, so the batch of 16 is split 8/8 across both cards; both GPUs report 100 % utilisation for a single training process. Forced alignment ran in a separate MFA 3.4.3 environment installed from conda-forge, invoked as `PATH=<env>/bin:$PATH <env>/bin/mfa ...` because a session restore had stripped the environment's exec bits and shared-library symlinks.

Appendix A — measured figures used above

Quantity	English	Telugu
Clips in manifests	5,582	3,363
Usable clips	5,322 (4,500 / 822)	3,269 (2,751 / 518)
Usable hours	4.842	4.700
Speakers	2	1
Dropped	260	94
Params	35,159,617	35,159,617
Vocoder	HiFi-GAN universal (pretrained)	same